

说明

串行外设接口（SPI）是一种常用的同步串行通信协议，广泛用于连接传感器、存储器、显示器等外设。SPI 通信中，主设备通过片选信号 NSS 选择从设备进行通信。

本应用笔记重点介绍如何在 SPI 主机模式下使用 NSS 输出功能，以提高通信可靠性。

适用范围

适用范围	系列
通用 MCU	CH32V103 CH32V203 CH32V307 CH32L103 ...

目录

说明	i
目录	ii
表格索引	ii
图片索引	ii
第 1 章 使用硬件控制 NSS	1
1.1 硬件 NSS 工作原理	1
1.2 硬件 NSS 配置步骤	1
1.2.1 硬件连接	1
1.2.1 软件代码	1
第 2 章 使用软件控制 NSS	4
2.1 软件 NSS 的介绍	4
2.2 软件 NSS 配置步骤	4
2.2.1 硬件连接	4
2.2.1 软件代码	4
2.3 软件 NSS 的优点	6
第 3 章 常用 CH32 芯片的 SPI 速度	7
3.1 常用 CH32 芯片最大速度	7
历史版本	8
声明	9

表格索引

表 1-1 硬件 NSS 的 SPI1 硬件连接	1
表 2-1 软件 NSS 的 SPI1 硬件连接	4
表 3-1 常用 CH32 的 SPI 最大速度表	7

图片索引

图 1-1 硬件 NSS 通信示意图	1
图 2-1 软件 NSS 通信示意图	4

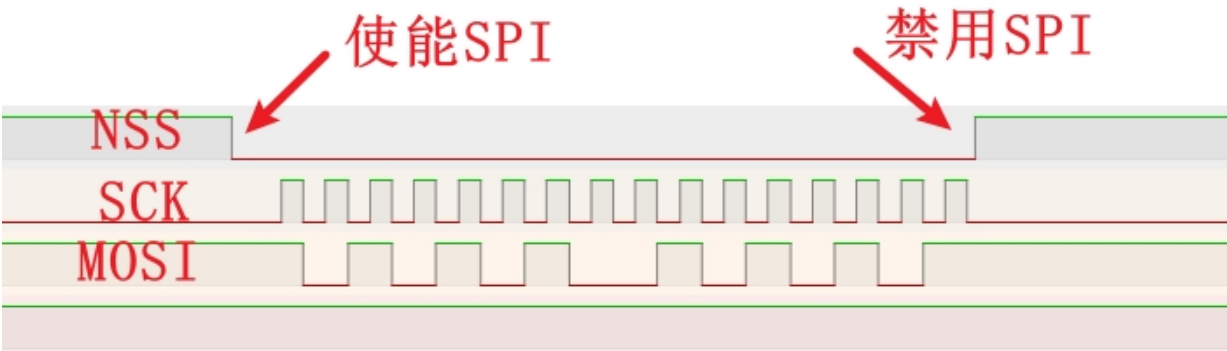
第1章 使用硬件控制 NSS

1.1 硬件 NSS 工作原理

当 SPI 配置为主机且启用了硬件 NSS 输出时：

- NSS 引脚在 SPI 使能（SPE = 1）后自动拉低（有效电平，通常为低电平）。
- 每次 SPI 传输开始，NSS 保持低电平。
- 若 SPI 被禁用（SPE = 0），NSS 将恢复为高电平。

图 1-1 硬件 NSS 通信示意图



1.2 硬件 NSS 配置步骤

1.2.1 硬件连接

在使用 SPI1 时，使用 PA4 连接目标的 NSS 引脚提供功能，具体连接如表格所示。

表 1-1 硬件 NSS 的 SPI1 硬件连接

引脚	功能	目标
PA5	SPI1_SCK	从机 SCK
PA6	SPI1_MISO	从机 MISO
PA7	SPI1_MOSI	从机 MOSI
PA4	SPI1_NSS	从机 NSS（低有效）

1.2.1 软件代码

配置时，先启用 SPI1 和 GPIO；配置 PA4、PA5、PA7 为复用功能 GPIO_Mode_AF_PP，将 PA6 配置 GPIO_Mode_IN_FLOATING。然后将 SPI 模式设置为 2Lines_FullDuplex，并开启硬件 NSS 功能，然后开启 NSS 输出功能。注意此时不要使能 SPI。

```
void SPI_FullDuplex_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    SPI_InitTypeDef SPI_InitStructure = {0};
```

```

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_SPI1,
ENABLE);

    GPIO_InitStructure.GPIO_Pin    = GPIO_Pin_4;
    GPIO_InitStructure.GPIO_Mode    = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed  = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin    = GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode    = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed  = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin    = GPIO_Pin_6;
    GPIO_InitStructure.GPIO_Mode    = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin    = GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Mode    = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed  = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    SPI_InitStructure.SPI_Direction    =
SPI_Direction_2Lines_FullDuplex;
    SPI_InitStructure.SPI_Mode          = SPI_Mode_Master;
    SPI_InitStructure.SPI_DataSize      = SPI_DataSize_16b;
    SPI_InitStructure.SPI_CPOL          = SPI_CPOL_Low;
    SPI_InitStructure.SPI_CPHA          = SPI_CPHA_1Edge;
    SPI_InitStructure.SPI_NSS           = SPI_NSS_Hard;    // 使用硬件
NSS
    SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_64;
    SPI_InitStructure.SPI_FirstBit       = SPI_FirstBit_MSB;
    SPI_InitStructure.SPI_CRCPolynomial  = 7;
    SPI_Init(SPI1, &SPI_InitStructure);

    // 开启 SPI 的 NSS 输出功能
    SPI_SSOutputCmd(SPI1, ENABLE);
}

```

使用时，需要先使能 SPI 再进行发送接收，此时硬件 NSS 会拉低 NSS 的 IO，然后再开始发送；在结束时，需要先等待 SPI 当前的流程完成再关闭 SPI，此时硬件 NSS 会拉高 NSS 的 IO，结束流程。

```

// 使能 SPI NSS 拉低
SPI_Cmd(SPI1, ENABLE);

while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_TXE) == RESET)
{
}

SPI_I2S_SendData(SPI1, 0xaa55);

```

```
while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_RXNE) == RESET)
{
}

SPI_I2S_ReceiveData(SPI1);

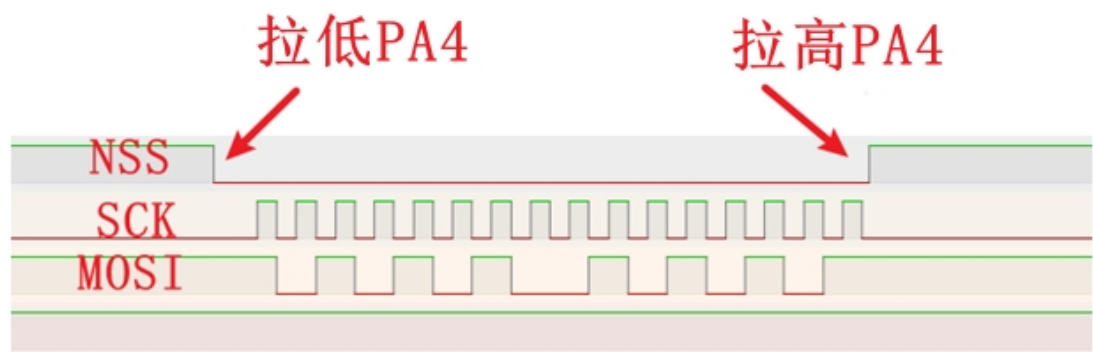
// 禁用 SPI NSS 拉高
SPI_Cmd(SPI1, DISABLE);
```

第2章 使用软件控制 NSS

2.1 软件 NSS 的介绍

- 在许多实际应用中，尤其是多从机系统的场景，软件控制 NSS（也称为“GPIO 方式控制 NSS”）是更常用且灵活的选择。

图 2-1 软件 NSS 通信示意图



2.2 软件 NSS 配置步骤

2.2.1 硬件连接

在使用软件 NSS 的 SPI1 时，仍然可以使用 PA4 连接目标的 NSS 引脚提供功能，此时使用软件对 NSS 进行管理，具体连接如表格所示。

表 2-1 软件 NSS 的 SPI1 硬件连接

引脚	功能	目标
PA5	SPI1_SCK	从机 SCK
PA6	SPI1_MISO	从机 MISO
PA7	SPI1_MOSI	从机 MOSI
PA4	GPIO_Output	从机 NSS（低有效）

2.2.1 软件代码

配置时，先启用 SPI1 和 GPIO；将 PA4 配置 GPIO_Mode_Out_PP，并设置输出高，配置 PA5、PA7 为复用功能 GPIO_Mode_AF_PP，将 PA6 配置 GPIO_Mode_IN_FLOATING。然后将 SPI 模式设置为 2Lines_FullDuplex，并使用软件 NSS。

```
void SPI_FullDuplex_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    SPI_InitTypeDef SPI_InitStructure = {0};

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_SPI1, ENABLE);
```

```

GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_4;
GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_Out_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure);
GPIO_SetBits(GPIOA, GPIO_Pin_4);

GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_5;
GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_AF_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure);

GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_6;
GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_IN_FLOATING;
GPIO_Init(GPIOA, &GPIO_InitStructure);

GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_7;
GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_AF_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure);

SPI_InitStructure.SPI_Direction   = SPI_Direction_2Lines_FullDuplex;
SPI_InitStructure.SPI_Mode       = SPI_Mode_Master;
SPI_InitStructure.SPI_DataSize   = SPI_DataSize_16b;
SPI_InitStructure.SPI_CPOL      = SPI_CPOL_Low;
SPI_InitStructure.SPI_CPHA      = SPI_CPHA_1Edge;
SPI_InitStructure.SPI_NSS       = SPI_NSS_Soft; // 使用软件 NSS
SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_64;
SPI_InitStructure.SPI_FirstBit  = SPI_FirstBit_MSB;
SPI_InitStructure.SPI_CRCPolynomial = 7;
SPI_Init(SPI1, &SPI_InitStructure);
}

```

使用时，需要先拉低 PA4，此时 NSS 被拉低，然后再开始发送；
在结束时，需要先等待 SPI 当前的流程完成再拉高 PA4，此时 NSS 被拉高，结束流程。

```

// 拉低 PA4 NSS 拉低
GPIO_ResetBits(GPIOA, GPIO_Pin_4);

while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_TXE) == RESET)
{
}

SPI_I2S_SendData(SPI1, 0xaa55);

while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_RXNE) == RESET)
{
}

```

```
SPI_I2S_ReceiveData(SPI1);  
  
// 拉高 PA4 NSS 拉高  
GPIO_ResetBits(GPIOA, GPIO_Pin_4);
```

2.3 软件 NSS 的优点

软件 NSS 相对于硬件 NSS 有一些优点：

- 支持多从机，可通过多个 GPIO 分别控制不同从设备，轻松扩展系统。
- 时序完全可控，可在任意时刻拉低/拉高 NSS，满足特殊协议要求（如需 NSS 在整个事务期间保持低电平）。
- 兼容性好，适用于所有 SPI 从设备，无论其对 NSS 时序是否有特殊要求。
- 无硬件限制，不依赖特定 NSS 引脚，可使用任意 GPIO，布线更自由。

第3章 常用 CH32 芯片的 SPI 速度

3.1 常用 CH32 芯片最大速度

一般 CH32 芯片的 SPI 设计最大速度是对应的总线频率的一半，详细见下表。

表 3-1 常用 CH32 的 SPI 最大速度表

芯片系列	最大主频	最大 SPI 速度
CH32V317	144MHz	72MHz
CH32V30x	144MHz	72MHz
CH32V203	144MHz	72MHz
CH32V205	144MHz	72MHz
CH32V208	144MHz	72MHz
CH32V208	144MHz	72MHz
CH32V103	72MHz	36MHz
CH32V00x	48MHz	24MHz
CH32X035	48MHz	24MHz
CH32L103	96MHz	48MHz
CH32M030	72MHz	36MHz

历史版本

更新内容

日期	版本	变更内容
2025/11/17	V1.0	初版发行

声明

本手册版权所有为南京沁恒微电子股份有限公司（Copyright © Nanjing Qinheng Microelectronics Co., Ltd. All Rights Reserved），未经南京沁恒微电子股份有限公司书面许可，任何人不得因任何目的、以任何形式（包括但不限于全部或部分地向任何人复制、泄露或散布）不当使用本产品手册中的任何信息。

任何未经允许擅自更改本产品手册中的内容与南京沁恒微电子股份有限公司无关。

南京沁恒微电子股份有限公司所提供的说明文档只作为相关产品的使用参考，不包含任何对特殊使用目的的担保。南京沁恒微电子股份有限公司保留更改和升级本产品手册以及手册中涉及的产品或软件的权利。

参考手册中可能包含少量由于疏忽造成的错误。已发现的会定期勘误，并在再版中更新和避免出现此类错误。